
Diseño Arquitectónico

INFO248

Dra. Valeria Henríquez

Prueba de entrada 6 min



https://docs.google.com/forms/d/18CUJelMHBu_eFsoDwsoryhe77j95LfXZ6zJnQZf2x6l/edit

¿Qué es el diseño arquitectónico?

El diseño arquitectónico se interesa por entender **cómo debe organizarse un sistema y cómo tiene que diseñarse la estructura global** de ese sistema.

Un modelo arquitectónico **describe la forma en que se organiza un sistema** como un conjunto de componentes en comunicación.

Analogía: Pizzería

Frontend = el mesero que recibe el pedido.

Backend = la cocina que procesa lo pedido.

Base de datos = la libreta secreta con las recetas, clientes frecuentes, precios, etc.

Lógica de negocios = la caja que sabe de descuentos, ofertas, etc.

Servicio de envíos = los repartidores.

Cola de mensajes = la fila de comandas pegadas en la cocina.

Monitoreo = el gerente mirando si todo va bien o si alguien quemó 40 pizzas.

“Un sistema mal diseñado es como una pizzería donde el cocinero también atiende el teléfono, cobra, maneja la moto y habla con los clientes en redes sociales.”

¿Por qué importa la arquitectura? no se trata solo de que el software funcione, sino de que soporte crecimiento, cambios, errores y evite el caos.

Niveles de abstracción de la arquitecturas de software

La arquitectura en pequeño se interesa por la arquitectura de sistemas individuales. En este nivel, uno se preocupa por la forma en que el **sistema individual se separa en componentes**.

Ejemplos: MVC, Arquitectura de capas, Arquitectura cliente-servidor, Arquitectura de tubería y filtro, Pizarra, Peer-to-peer, etc.

La arquitectura en grande se interesa por la arquitectura de sistemas empresariales complejos que incluyen otros sistemas, programas y componentes de programa.

Ejemplos: Sistemas distribuidos, Microservicios, Serverless, orientada a servicios

¿Qué entendemos por patrones de diseño?

Un patrón de diseño resulta ser una **solución a un problema de diseño**. Para que una solución sea considerada un patrón debe poseer ciertas características.

- Haber **comprobado su efectividad** resolviendo problemas similares en ocasiones anteriores.
- Debe ser **reutilizable**, lo que significa que es aplicable a diferentes problemas de diseño en distintas circunstancias

¿Qué entendemos por patrones de diseño?

Un patrón arquitectónico es una colección de decisiones de diseño arquitectónicas que tiene un nombre específico y que son aplicables a problemas de diseño recurrentes, y son parametrizadas para tener en cuenta diferentes contextos de desarrollo de software en los cuales el problema aparece.

Taylor, Medvidovic and Dashofy

No confundir

Patrones de diseño arquitectónico. Aquellos que expresan un esquema organizativo estructural fundamental para sistemas de software. Origen: "Pattern-Oriented Software Architecture"

Patrones de diseño de software. Abstracciones generales que ocurren a través de las aplicaciones que muestran objetos e interacciones abstractas y concretas. Origen: "Design Patterns: Elements of Reusable Object-Oriented Software"

Dialectos (idioms): Patrones de bajo nivel específicos de un lenguaje de programación o entorno concreto

Ejemplos: Arquitectura de microservicios
Arquitectura en capas
Modelo Vista Controlador (MVC)
Arquitectura orientada a servicios.

Sistema

Ejemplos: Singleton (creacional)
Facade (estructural) Observer (comportamental) Strategy (comportamental), principios SOLID

Programas

Ejemplos:
Iterar con índice y valor: Un idiom muy usado en Python:

```
for i, valor in enumerate(lista):  
    print(i, valor)
```

Default value idiom: Asignar un valor por defecto si una variable viene vacía o nula.

```
const nombre = input || "Anónimo";
```

Null check / safe access: Acceder solo si existe el objeto.

```
const ciudad = usuario?.direccion?.ciudad;
```

¿Qué se busca al modelar una arquitectura?

Alta cohesión (una única responsabilidad) y Bajo acoplamiento (mínima cantidad de dependencias)

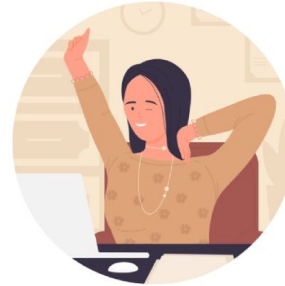
¿Por qué buscamos eso?

La buena arquitectura es algo que sustenta la evolución y está profundamente entrelazada con la velocidad de incorporar nuevas funcionalidades y corregir problemas

¿Qué queremos evitar? - Design Smells

Síntoma	Definición	Consecuencias
Rigidez	Dificultad de realizar cambios en el software, incluso en tareas sencillas.	• Las estimaciones son más abultadas. • Añadir funcionalidades nuevas cuesta mucho cuando antes era más rápido.
Fragilidad	Tendencia a que el software se rompa en múltiples sitios cada vez que se realiza un cambio en él, incluso sin tener relación el cambio con lo que se rompe.	• Subir una nueva versión y que se rompan partes que aparentemente no tienen que ver con lo modificado. • Aparición de mayor cantidad de bugs, aumentando el coste de esfuerzo y tiempo en corregirlos.
Inmovilidad	Incapacidad de reutilizar piezas de código por el gran acoplamiento que tiene, haciendo que este sea inamovible cuando la funcionalidad es prácticamente la misma que la deseada.	• Aumenta la complejidad del diseño y del código al introducir código duplicado. • Complica el entendimiento del código y genera equivocaciones.
Viscosidad	La tendencia de que, en el ámbito del software, realizar la solución buena requiere mayor esfuerzo comparado con la solución mala y rápida y, en el ámbito del entorno de desarrollo, éste es lento e ineficiente.	• Provoca el efecto bola de nieve: cuanto más tarde se corrija esa deuda técnica más difícil y costoso será resolverlo. • Pérdida de tiempo, estrés, confusión, pasotismo, acciones inseguras, etc.

5 Min. de Pausa





Arquitectura multicapa

<https://martinfowler.com/bliki/PresentationDomainDataLayering.html>

¿Qué es?

En inglés **Multi-Layer Architecture** y como su propio nombre indica, refleja la **separación lógica en capas** de un sistema software. Una capa es simplemente, un conjunto de clases o paquetes que tienen unas responsabilidades relacionadas dentro del funcionamiento del sistema.



¿EN QUÉ CONSISTE?

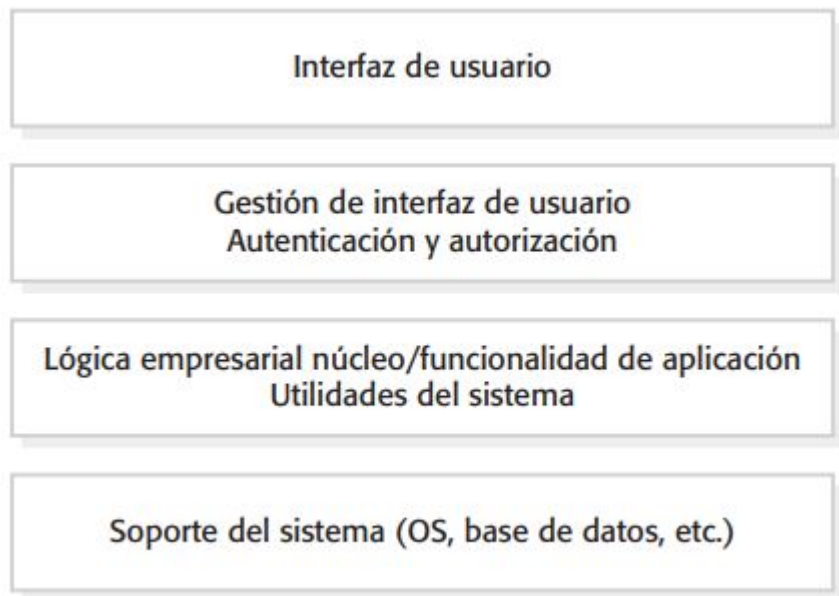
Las capas en esta arquitectura **están organizadas de forma jerárquica** unas encima de otras y las dependencias siempre van hacia al interior. Es decir, **una capa concreta dependerá solamente de las capas inferiores pero nunca de las superiores**. Las capas más comunes son **Presentación, Aplicación, Dominio de negocio y Acceso o Persistencia de datos**. La comunicación entre capas se puede hacer aplicando el principio de **Inversión de Dependencias (Dependency Inversion)** para reducir el acoplamiento y facilitar el testing.

- **Capa de presentación:** es la encargada de gestionar la interacción que tiene el cliente con nuestra aplicación. Actúa como mediador.
- **Capa de aplicación:** contiene los casos de uso que implementan la lógica de negocio. Es decir, el conjunto de reglas (establecidas por la organización) sobre las que se rige nuestra aplicación a la hora de ejecutarse.
- **Capa de Dominio:** contiene las entidades de negocio.
- **Capa de Datos:** su tarea principal es la persistencia y recuperación de los datos.

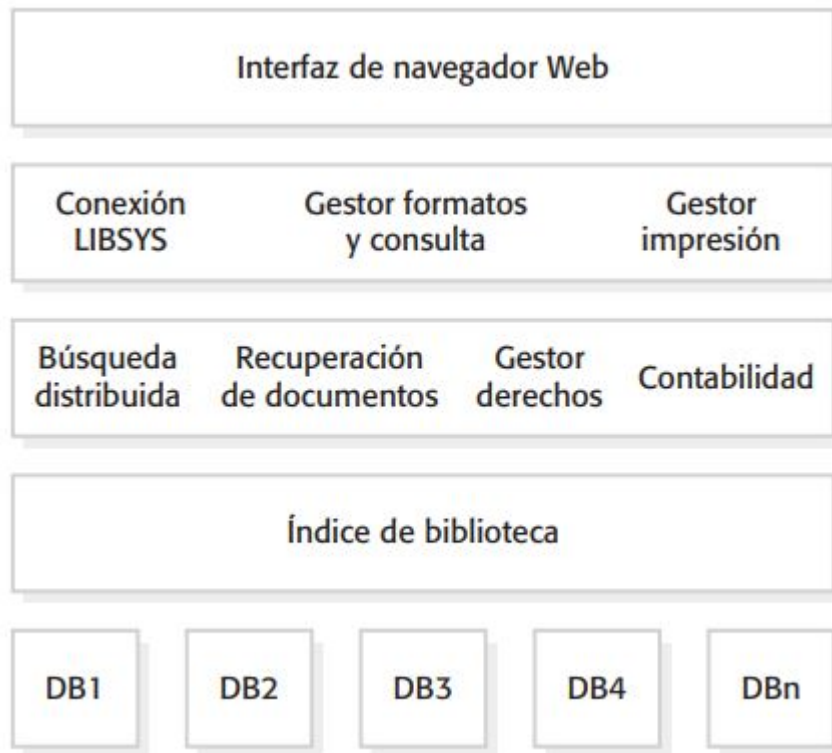
Este es un ejemplo de arquitectura multicapa con 4 niveles pero dependiendo de cada organización y de las necesidades de negocio, puede haber variaciones.



Arquitectura en capas genérica

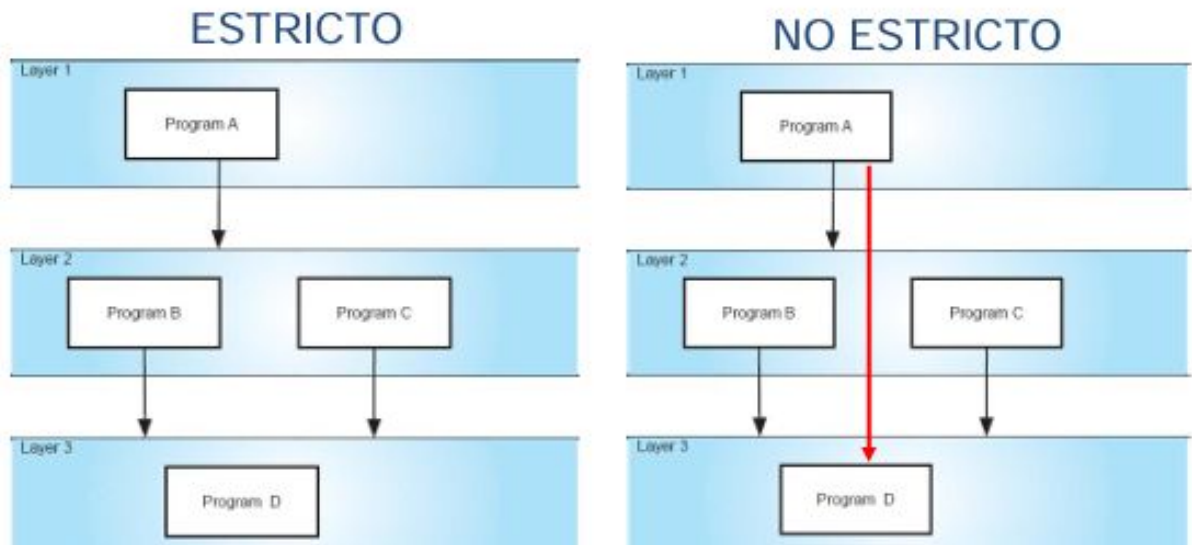


Arquitectura del sistema LIBSYS



CAPAS

Una capa ofrece un conjunto de servicios (“interface” o API). Dichos servicios pueden ser accedidos por programas que residen en la capa superior. La encapsulación de la implementación de los servicios de la capa puede ser de 2 modos: estricto y no estricto



Resumen - Arquitectura de capas

Nombre	Arquitectura en capas
Descripción	Organiza el sistema en capas con funcionalidad relacionada con cada capa. Una capa da servicios a la capa de encima, de modo que las capas de nivel inferior representan servicios núcleo que es probable se utilicen a lo largo de todo el sistema. Véase la figura 6.6.
Ejemplo	Un modelo en capas de un sistema para compartir documentos con derechos de autor se tiene en diferentes bibliotecas, como se ilustra en la figura 6.7.
Cuándo se usa	Se usa al construirse nuevas facilidades encima de los sistemas existentes; cuando el desarrollo se dispersa a través de varios equipos de trabajo, y cada uno es responsable de una capa de funcionalidad; cuando exista un requerimiento para seguridad multinivel.
Ventajas	Permite la sustitución de capas completas en tanto se conserve la interfaz. Para aumentar la confiabilidad del sistema, en cada capa pueden incluirse facilidades redundantes (por ejemplo, autenticación).
Desventajas	En la práctica, suele ser difícil ofrecer una separación limpia entre capas, y es posible que una capa de nivel superior deba interactuar directamente con capas de nivel inferior, en vez de que sea a través de la capa inmediatamente abajo de ella. El rendimiento suele ser un problema, debido a múltiples niveles de interpretación de una solicitud de servicio mientras se procesa en cada capa.

Actividad

Acepte la tarea <https://classroom.github.com/a/5nWbU6LK> y vaya a la carpeta “3 capas en python”:

- ¿Qué elementos identifica?
- Cree el diagrama de componentes
- Realice el seguimiento de una operación y cree el diagrama de secuencia.
- Imagine una nueva funcionalidad para el sistema.
- Cree el diagrama de secuencia para implementar dicha funcionalidad
- Implemente la funcionalidad siguiendo el patrón arquitectónico propuesto

Utilice IA para realizar esta tarea, verifique, cuestione y critique los resultados. Incluya esta información en el entregable.

